PropagateUnbackedSymInts#

https://pytorch.org/docs/stable/generated/torch.fx.experimental.symbolic_sha

Description:

Run module via interpretation and return the result. This uses the “boxed” calling convention, where you pass a list of arguments, which will be cleared by the interpreter. This ensures that input tensors are promptly deallocated. Backwards-compatibility for this API is guaranteed. Execute a call_function node and return the result. target (Target) – The call target for this node. See Node for details on semantics

Function Signature

```
class
torch.fx.experimental.symbolic_shapes.PropagateUnbackedSymIn
ts(module, garbage_collect_values=True, graph=None)[source]#

boxed_run(args_list)[source]#

call_function(target, args, kwargs)[source]#

Parameters

Return type
```

Returns:

Union[Tensor, tuple[torch.Tensor, …]]

Notes & Warnings

NOTE Note Backwards-compatibility for this API is guaranteed.

NOTE Note Backwards-compatibility for this API is guaranteed.

NOTE Note Backwards-compatibility for this API is guaranteed.

NOTE Note Backwards-compatibility for this API is guaranteed.

NOTE Note Backwards-compatibility for this API is guaranteed.

NOTE Note Backwards-compatibility for this API is guaranteed.

NOTE Note Backwards-compatibility for this API is guaranteed.

NOTE Note Backwards-compatibility for this API is guaranteed.

NOTE Note Backwards-compatibility for this API is guaranteed.

NOTE Note Backwards-compatibility for this API is guaranteed.

NOTE Note Backwards-compatibility for this API is guaranteed.

Detailed Documentation

Documentation

Run module via interpretation and return the result. This uses the “boxed” calling convention, where you pass a list of arguments, which will be cleared by the interpreter. This ensures that input tensors are promptly deallocated.

Backwards-compatibility for this API is guaranteed.

Execute a call_function node and return the result.

target (Target) – The call target for this node. See Node for details on semantics

args (Tuple) – Tuple of positional args for this invocation

kwargs (Dict) – Dict of keyword arguments for this invocation

Any: The value returned by the function invocation

Backwards-compatibility for this API is guaranteed.

Execute a call_method node and return the result.

target (Target) – The call target for this node. See Node for details on semantics

args (Tuple) – Tuple of positional args for this invocation

kwargs (Dict) – Dict of keyword arguments for this invocation

Any: The value returned by the method invocation

Backwards-compatibility for this API is guaranteed.

Execute a call_module node and return the result.

target (Target) – The call target for this node. See Node for details on semantics

args (Tuple) – Tuple of positional args for this invocation

kwargs (Dict) – Dict of keyword arguments for this invocation

Any: The value returned by the module invocation

Backwards-compatibility for this API is guaranteed.

Fetch the concrete values of args and kwargs of node n from the current execution environment.

n (Node) – The node for which args and kwargs should be fetched.

args and kwargs with concrete values for n.

Backwards-compatibility for this API is guaranteed.

Fetch an attribute from the Module hierarchy of self.module.

target (str) – The fully-qualified name of the attribute to fetch

The value of the attribute.

Backwards-compatibility for this API is guaranteed.

Execute a get_attr node. Will retrieve an attribute value from the Module hierarchy of self.module.

target (Target) – The call target for this node. See Node for details on semantics

args (Tuple) – Tuple of positional args for this invocation

kwargs (Dict) – Dict of keyword arguments for this invocation

The value of the attribute that was retrieved

Backwards-compatibility for this API is guaranteed.

Recursively descend through args and look up the concrete value for each Node in the current execution environment.

args (Argument) – Data structure within which to look up concrete values

n (Node) – Node to which args belongs. This is only used for error reporting.

Optional[Union[tuple['Argument', …], Sequence[Argument], Mapping[str, Argument], slice, range, Node, str, int, float, bool, complex, dtype, Tensor, device, memory_format, layout, OpOverload, SymInt, SymBool, SymFloat]]

Backwards-compatibility for this API is guaranteed.

Execute an output node. This really just retrieves the value referenced by the output node and returns it.

target (Target) – The call target for this node. See Node for details on semantics

args (Tuple) – Tuple of positional args for this invocation

kwargs (Dict) – Dict of keyword arguments for this invocation

The return value referenced by the output node

Backwards-compatibility for this API is guaranteed.

Execute a placeholder node. Note that this is stateful: Interpreter maintains an internal iterator over arguments passed to run and this method returns next() on that iterator.

target (Target) – The call target for this node. See Node for details on semantics

args (Tuple) – Tuple of positional args for this invocation

kwargs (Dict) – Dict of keyword arguments for this invocation

The argument value that was retrieved.

Backwards-compatibility for this API is guaranteed.

Run module via interpretation and return the result.

*args – The arguments to the Module to run, in positional order

initial_env (Optional[Dict[Node, Any]]) – An optional starting environment for execution. This is a dict mapping Node to any value. This can be used, for example, to pre-populate results for certain Nodes so as to do only partial evaluation within the interpreter.

enable_io_processing (bool) – If true, we process the inputs and outputs with graph's process_inputs and process_outputs function first before using them.

The value returned from executing the Module

Backwards-compatibility for this API is guaranteed.

Run an FX node, propagating unbacked Symbol bindings to the new fake tensor

Union[Tensor, tuple[torch.Tensor, …]]